

BOOK RECOMMENDER

Matteo De Nicola	757933
Valerio Siragusa	756227
Alessandro Saccon	756145
Luca Brogginì	756446
Andrea Pianezzola	756141

1. Introduzione	2
2. Tecnologie utilizzate	2
3. Organizzazione directory	2
4. Progettazione software.....	3
4.1 Architettura del sistema	3
4.2 Classi e package.....	4
4.2.1 Package App:	4
4.2.2 Package Client:.....	5
4.2.3 Package Server:.....	5
4.2.4 Package Model:	6
4.2.5 Package View:.....	9
4.3 Diagrammi UML	15
4.3.1 Class diagram	15
4.3.2 Sequence diagram della ricerca per titolo	16
5. Progettazione database.....	17
5.1 Diagramma ER.....	17
5.2 Architettura database.....	18
5.3 Schema relazionale.....	18
6. Limiti della soluzione sviluppata	19
6.1 Limiti funzionali e di sicurezza	19
6.1.1 Memorizzazione Password in chiaro	19
6.1.2 Credenziali hardcoded	19
6.1.3 Mancanza di caching	19
6.2 Limiti di usabilità.....	19
6.2.1 Messaggi di errore generici	19
6.2.2 Assenza di validazione client-side.....	19
6.2.3 Interfaccia non-responsive	19
7. Sitografia e bibliografia	20

1. Introduzione

Book Recommender è un progetto sviluppato nell'ambito del progetto di Laboratorio A e B per il corso di laurea in Informatica dell'Università degli Studi dell'Insubria.

Il progetto "Book Recommender" prevede la creazione di un sistema per la valutazione e raccomandazione di libri. Gli utenti registrati possono creare librerie personali, recensire i libri secondo criteri specifici, e suggerire libri correlati. Tutti gli utenti, registrati e non, possono consultare il repository di libri, cercare opere per titolo, autore o anno, e visualizzare le valutazioni e i consigli aggregati per ciascun libro.

Le funzionalità principali includono la registrazione degli utenti, la gestione delle librerie personali, e la consultazione del repository di libri con la possibilità di visualizzare valutazioni e suggerimenti. I dati relativi ai libri e agli utenti sono memorizzati in file di testo strutturati.

Il progetto è sviluppato in Java 17, ed è stato sviluppato e testato sul sistema operativo Windows 11 e su macOS.

2. Tecnologie utilizzate

Linguaggio: Java 17

Framework UI: JavaFX 20.0.2

Database: PostgreSQL

Build Tool: Apache Maven 3

Testing: JUnit 5.11.0

Driver JDBC: PostgreSQL JDBC 42.7.8

3. Organizzazione directory

```
book-recommender/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── app/
│   │   │   │   └── Bookapp.java
│   │   │   ├── client/
│   │   │   │   └── ClientService.java
│   │   │   ├── model/
│   │   │   │   ├── Libro.java
│   │   │   │   ├── Utente.java
│   │   │   │   ├── Suggerimento.java
│   │   │   │   ├── Valutazione.java
│   │   │   │   ├── RepositoryLibri.java
│   │   │   │   └── SERVERBOOK.java
│   │   │   ├── server/
│   │   │   │   └── DATABASE.java
│   │   │   └── view/
│   │   │       ├── loginController.java
│   │   │       ├── registrazioneController.java
│   │   │       └── repositoryController.java
│   │   └── resources/
│   │       ├── view/style.css
│   │       └── schema.sql
```

4. Progettazione software

4.1 Architettura del sistema

Il sistema BookRecommender è progettato con un'architettura a livelli, secondo il modello Client-Server. L'interfaccia utente è gestita da componenti JavaFX distribuiti nei package `app` e `view`, dove la classe principale coordina le diverse schermate come login, registrazione e consultazione del repository. La logica applicativa risiede nel package `model`, che definisce le entità di dominio quali libri, utenti e suggerimenti, insieme alle regole di business e alle validazioni necessarie.

Per l'accesso ai dati, le classi modello contengono metodi statici che utilizzano query sicure tramite `PreparedStatement` per interagire con un database PostgreSQL, la cui configurazione e gestione vengono affidate al package `server`. La comunicazione tra client e server avviene via socket TCP/IP, garantendo uno scambio di dati serializzati efficiente e sicuro.

Questa architettura modulare favorisce una chiara organizzazione del codice, mantenendo separati gli aspetti grafici, la logica applicativa e la gestione dei dati. Segue ora una scaletta che illustra i vari livelli che abbiamo utilizzato per la creazione del progetto:

- **Livello di Presentazione**
Package: `app`, `view`
Componenti:
 - `Bookapp.java`: classe principale dell'applicazione JavaFX
 - `Controllers`: `loginController`, `registrazioneController`, `repositoryController`
 - `style.css`: foglio di stile per l'interfaccia utenteResponsabilità:
 - Gestione dell'interfaccia utente
 - Handling degli eventi utente
 - Navigazione tra le scene
 - Rendering dei componenti grafici
- **Livello Logico**
Package: `model`
Componenti:
 - `Libro.java`
 - `Utente.java`
 - `Suggerimento.java`
 - `Valutazione.java`
 - `RepositoryLibri.java`Responsabilità:
 - Implementazione della logica applicativa
 - Validazione dei dati
 - Regole di business
 - Gestione delle entità di dominio
- **Livello di Accesso ai Dati**
Package: `model` (metodi statici per operazioni database)
Componenti:
 - Metodi statici nelle classi `model` per interazione con PostgreSQL
 - Utilizzo di `PreparedStatement` per sicurezza

Responsabilità:

- Esecuzione query SQL
- Mapping tra oggetti Java e tabelle database
- Gestione connessioni JDBC

- Livello Dati

Package: server

Componenti:

- DATABASE.java: inizializzazione e configurazione database
- PostgreSQL DBMS
- schema.sql: definizione schema database

Responsabilità:

- Persistenza dei dati
- Integrità referenziale
- Ottimizzazione tramite indici

4.2 Classi e package

4.2.1 Package App:

4.2.1.1 Classe Bookapp:

Bookapp è la classe principale dell'applicazione, e gestisce il ciclo di vita dell'interfaccia grafica JavaFX. Si occupa di creare la finestra principale, inizializzare la connessione al server tramite ClientService, impostare il layout di base, e controllare la navigazione tra le diverse scene (home, login, registrazione, repository). Mantiene inoltre lo stato dell'utente autenticato. La classe contiene anche metodi per avviare e chiudere correttamente l'applicazione, incluse operazioni di pulizia e chiusura della connessione al server.

Metodi Principali:

- start(Stage primaryStage) Inizializza la finestra principale (primaryStage), la massimizza, crea un'istanza di ClientService e tenta di connettersi al server sulla porta 8080. Se la connessione fallisce, stampa un errore e ferma l'esecuzione. In caso positivo, inizializza il layout root e mostra la schermata home. Complessità: $O(1)$, operazioni fisse di inizializzazione.
- initRootLayout() Inizializza il layout principale usando un BorderPane, crea una nuova scena con questo layout, carica il file CSS dallo e lo applica alla scena. Imposta la scena nel primaryStage e la mostra. Gestisce errori se il file CSS non viene trovato.
- showHomeView() Costruisce la schermata principale home con tre pulsanti: "Consulta libreria", "Registrazione" e "Login". Applica stili CSS e imposta i gestori di evento per la navigazione verso le schermate relative. Organizza i componenti in un VBox e lo inserisce al centro del layout.
- showLogin() Istanza il controller loginController passando l'istanza corrente e il ClientService per gestire il login.
- showRegistrazione() Simile a showLogin(), istanzia registrazioneController per la schermata di registrazione.
- showRepository() Istanza il controller repositoryController, che gestisce la visualizzazione e interazione col repository dei libri.
- showMainMenuScene() Ritorna alla schermata home chiamando showHomeView().
- setLoggedUser(Utente user) Imposta quale utente è attualmente autenticato nel sistema.
- getLoggedUser() Restituisce l'oggetto Utente attualmente loggato o null se nessuno è autenticato.

- `getLoggedUserId()` Restituisce l'ID numerico dell'utente loggato o -1 se non è presente nessun utente.
- `stop()` Metodo chiamato alla chiusura dell'applicazione. Chiude la connessione al server tramite `clientService.close()` se la connessione è attiva.

Tutti i metodi della classe `Bookapp` hanno complessità costante $O(1)$, perché si occupano principalmente di operazioni di setup, UI building, gestione di istanze di controller, e operazioni semplici di accesso e modifica dello stato interno (utente loggato e connessione client). Non sono coinvolti algoritmi di ricerca o calcoli complessi in questa classe: rappresenta la parte di coordinamento e controllo dell'applicazione.

4.2.2 Package Client:

4.2.2.1 Classe *ClientService*:

`ClientService` è la classe responsabile della comunicazione client-server tramite socket TCP/IP. Gestisce la connessione con il server, l'invio di richieste serializzate e la ricezione di risposte serializzate (oggetti Java). Espone metodi per effettuare tutte le operazioni di business come ricerche, login, registrazione, gestione librerie, inserimento valutazioni e suggerimenti. Metodi principali con descrizione e complessità:

- `connect(String host, int port)` Apre una connessione socket verso l'indirizzo `host` sulla porta `port`, inizializza gli stream di input/output per la serializzazione.
- `close()` Chiude la connessione socket se aperta.
- `ricercaPerTitolo(String titolo)` Invia la richiesta "RicercaPerTitolo" con il parametro `titolo` al server. Riceve e ritorna una lista di libri corrispondenti.
- `ricercaPerAutore(String autore)` Simile a `ricercaPerTitolo` ma ricerca libri per autore.
- `ricercaPerAutoreAnno(String autore, String anno)` Invia richiesta per ricerca combinata autore + anno di pubblicazione.

La classe `ClientService` contiene molti altri metodi che come `ricercaPerAutore` e `ricercaPerAutoreAnno` si limitano a inviare richieste. Tutti i metodi presenti in questa classe hanno complessità costante $O(1)$ lato client, in quanto si limitano a serializzare e inviare richieste, e ricevere risposte dal server. Il vero lavoro computazionale e la complessità algoritmica sono delegati al server.

4.2.3 Package Server:

4.2.3.1 Classe *DATABASE*:

La classe `DATABASE`, facente parte del package `server`, gestisce l'inizializzazione e configurazione del database PostgreSQL utilizzato dall'applicazione `BookRecommender`. Il suo scopo principale è controllare l'esistenza del database, crearlo se necessario, ed eseguire lo script di creazione dello schema SQL per preparare le tabelle e le relazioni.

Metodoi principali:

- `initialize()` Metodo che svolge la seguente sequenza di operazioni:
Stabilisce la connessione al database predefinito `postgres` usando le credenziali fornite. Esegue una query di controllo sull'esistenza di un database specifico denominato `bookrecommender`. Se il database `bookrecommender` non esiste, lo crea tramite il comando SQL `CREATE DATABASE`. Stampa un messaggio di conferma se il database è stato creato o se già esiste. Gestisce le eccezioni relative a problemi di connessione o permessi, con output di messaggi di errore. Tutte le operazioni di controllo e creazione del database sono $O(1)$, indipendenti da dati dinamici.

4.2.4 Package Model:

4.2.4.1 Classe Libreria:

La classe Libreria rappresenta una libreria personale di un utente nel sistema. Contiene attributi come userID (identificativo dell'utente proprietario) e nome (nome della libreria). Fornisce metodi per registrare una nuova libreria, controllarne l'esistenza, aggiungere libri alla libreria, ottenere le librerie di un utente e recuperare i libri contenuti in una libreria.

Metodi principali:

- `registraLibreria(int userID, String nomeLibreria)` Inserisce una nuova libreria nel database per l'utente specificato.
- `controllaLibreria(int userID, String nomeLibreria)` Verifica se la libreria esiste già per l'utente dato tramite query `SELECT EXISTS`.
- `aggiungiLibro(String nomeLibreria, int userID, int bookID)` Inserisce un'associazione tra una libreria, un utente e un libro nella tabella che mantiene i libri nelle librerie.
- `getLibrerieUtente(int userID)` Restituisce la lista di tutti gli oggetti Libreria associati all'utente specificato tramite query `SELECT`.
- `getElencoLibri(String nomeLibreria, int userID)` Restituisce la lista dei libri contenuti nella libreria indicata per l'utente dato, tramite `JOIN` tra `libri_libreria` e `libri`.
- `checkPresente(String nomeLibreria, int userID, int bookID)` Verifica se un libro è già presente in una libreria specifica di un utente, tramite query `SELECT EXISTS`.

Le operazioni di inserimento e controllo esistenza vengono eseguite con complessità costante $O(1)$ lato database grazie all'uso di indici appropriati. Le query di recupero liste (`getLibrerieUtente`, `getElencoLibri`) hanno complessità lineare $O(n)$ o $O(m)$ in base al numero di elementi restituiti.

4.2.4.2 Classe Libro:

La classe Libro, parte del package model, rappresenta un libro nel sistema BookRecommender. Include attributi quali ID, titolo, autori, categoria, editore e anno, e implementa l'interfaccia `Serializable` per consentire la serializzazione degli oggetti (utilizzata nella comunicazione client-server).

Metodi principali:

- `Libro(int ID, String titolo, String autori, String categoria, String editore, String anno)` Costruisce un oggetto libro con i dati specificati.
- `getID()`, `getTitolo()`, `getAutori()`, `getCategoria()`, `getEditore()`, `getAnno()` Restituiscono i rispettivi attributi del libro.
- `getLibro(String titolo)` Recupera dal database un libro corrispondente al titolo (query SQL con `ILIKE` con limite 1).
- `getLibro(int ID)` Recupera dal database un libro per ID (chiave primaria).
- `getValutazioni()` Recupera la lista delle valutazioni associate al libro.
- `getSuggerimenti()` Recupera la lista degli ID dei libri suggeriti associati al libro.
- `getInfo()` Restituisce una stringa contenente tutte le informazioni principali del libro (titolo, autori, ecc.).

Le operazioni di recupero dati basate su ID, grazie all'uso di indici su chiave primaria, hanno complessità costante $O(1)$. Le ricerche testuali basate su titolo utilizzando `ILIKE` hanno complessità variabile e possono oscillare tra $O(n \log n)$ e $O(n)$ a seconda della presenza e tipologia di indice. Le operazioni di caricamento liste di valutazioni e suggerimenti sono lineari rispetto al numero di elementi restituiti.

4.2.4.3 Classe RepositoryLibri:

RepositoryLibri è una classe del package model che funge da repository per le operazioni di ricerca e visualizzazione dei libri nel database. Espone metodi per effettuare ricerche nei campi titolo, autore e anno di pubblicazione, oltre a fornire una funzione per visualizzare dettagli specifici sui libri, incluse valutazioni e suggerimenti associati.

Metodi principali:

- ricercaPerTitolo(String titolo) Esegue una query SQL con ILIKE per cercare libri che abbiano il titolo contenente la stringa fornita. Ritorna una lista di oggetti Libro corrispondenti.
- ricercaPerAutore(String autore) Esegue una query SQL con ILIKE per trovare libri che contengano l'autore indicato. Restituisce una lista di libri trovati.
- LinkedList ricercaPerAutoreAnno(String autore, String anno) Esegue una query SQL con ILIKE combinando autore e anno di pubblicazione per filtrare i risultati. Ritorna una lista di libri corrispondenti.
- String visualizzaLibro(String titolo, int i) Recupera un libro tramite il titolo e fornisce una visualizzazione formattata basata sul parametro i :
i=1 : informazioni base del libro
i=2 : media valutazioni complessive, elenco libri suggeriti
i=3 : statistiche dettagliate per categorie di valutazione e numero di suggerimenti Ritorna una stringa

di output descrittiva.

Le query con operatori SQL ILIKE su titoli e autori hanno complessità tipica di $O(n)$ nel caso peggiore, dovuto alla scansione necessaria per il pattern matching senza indici specializzati.

La ricerca combinata autore-anno ha complessità simile, $O(n)$, con possibile ottimizzazione tramite indici composti.

Le operazioni di recupero e costruzione della lista libri richiedono tempo lineare rispetto al numero di risultati ottenuti ($O(m)$ con m numero risultati).

La visualizzazione dettagliata (case $i=2$ e $i=3$) può coinvolgere iterazioni su valutazioni e suggerimenti con complessità complessiva fino a $O(k\log n+m)$, dove k è il numero di suggerimenti e m il numero di valutazioni.

4.2.4.4 Classe SERVER-BOOK:

SERVER_BOOK è la classe principale che gestisce l'esecuzione del server BookRecommender. Si occupa di aprire una porta di ascolto TCP (porta 8080), inizializzare il database e le istanze delle classi modello (Libreria, Libro, Utente, Suggerimento, Valutazione, RepositoryLibri). Attende e accetta connessioni dei client, creando per ogni connessione un thread dedicato (serverThread_BOOK) che gestisce la comunicazione con il client. La classe contiene il metodo exec() che avvia il server e il metodo main() che lo esegue.

Metodi principali:

- exec() Inizializza il server socket sulla porta 8080. Esegue l'inizializzazione del database tramite DATABASE.initialize(). Crea istanze di tutte le entità modello per riutilizzo nel server. Entra in un ciclo infinito per accettare connessioni client. Per ogni connessione accettata, crea e avvia un thread serverThread_BOOK per gestire la comunicazione. Attende la terminazione del thread prima di chiudere la connessione (join). Gestisce eccezioni di I/O, interruzione thread, e socket.
- main(String[] args) Crea un'istanza di SERVER_BOOK ed esegue il metodo exec() per mettere in ascolto il server.

4.2.4.5 Classe serverThread_BOOK:

La classe serverThread_BOOK gestisce la comunicazione con un client connesso al server BookRecommender. Estende Thread e crea un thread separato per ogni client connesso. Si occupa di ricevere richieste serializzate, processarle invocando i metodi dei modelli (es. Utente, Libro, Libreria), restituire i risultati e gestire la connessione fino alla disconnessione del client.

La classe interpreta comandi come login, registrazione, ricerche, inserimenti di suggerimenti e valutazioni, aggiornamenti e query varie verso il database.

Metodi principali:

- Costruttore Inizializza lo stream di input/output e imposta la connessione socket con il client.
- run() Metodo principale del thread che esegue un ciclo di ascolto sulla socket del client, riceve i comandi, gestisce le operazioni secondo il comando ricevuto, e invia la risposta serializzata al client. Termina la comunicazione in caso di errore o disconnessione.

Il ciclo principale di run() è un loop di gestione richieste asincrone, con complessità $O(1)$ per ogni ciclo di ricezione-invio, in termini di overhead di rete e gestione socket. Le operazioni di elaborazione delle richieste coinvolgono chiamate ai metodi delle classi modello; la complessità complessiva dipende dai metodi chiamati (es. query SQL, manipolazioni dati). L'avvio e la gestione del thread sono $O(1)$ per ogni client. La gestione complessiva della concorrenza dipende dal numero di client connessi e dalle risorse del server; quindi, complessità e prestazioni sono legate al carico.

4.2.4.6 Classe Suggerimento:

La classe Suggerimento rappresenta un suggerimento fatto da un utente per un determinato libro. Contiene attributi quali ID suggerimento, ID utente e ID libro selezionato, e gestisce una lista degli eventuali libri suggeriti. Facilita l'inserimento di suggerimenti, associazione di libri suggeriti a un suggerimento, e il recupero di suggerimenti correlati ai libri.

Metodi principali:

- getCurrentID() Ottiene l'ultimo ID suggerimento inserito nel database tramite query `SELECT MAX(id)`.
- inserisciSuggerimento(int userID, int bookID) Inserisce un nuovo suggerimento nel database associando utente e libro.
- inserisciSuggerimentoLibro(int suggID, int bookID) Associa un libro suggerito a un suggerimento esistente nel database.
- getSuggerimentiLibro(int bookID) Recupera la lista di ID libri che sono stati suggeriti per un determinato libro (mediante join tra tabelle suggerimenti e libri associati).
- getNumeroConsigli(int userID, int bookID) Conta quante volte uno specifico utente ha suggerito un dato libro.

Le query principali di inserimento sono operazioni $O(1)$ costanti. Le operazioni di recupero e conteggio hanno complessità legata al numero di righe coinvolte nelle join, tipicamente $O(k \log n)$ con indici corretti, altrimenti $O(n*m)$.

4.2.4.7 Classe Utente:

La classe Utente rappresenta un utente registrato nel sistema BookRecommender. Contiene attributi come ID, nome, cognome, codice fiscale, email, userID e password. Offre metodi statici per la registrazione, autenticazione, validazione del codice fiscale e controllo dell'esistenza di utenti o codici fiscali nel sistema. Implementa anche l'interfaccia Serializable per permettere la serializzazione e la trasmissione dell'oggetto tra client e server.

Metodi principali:

- controlloCF(String cf) Valida che il codice fiscale rispetti il formato atteso (lunghezza 16, validità caratteri).
- esisteCF(String cf) Verifica se un codice fiscale è già presente nel database.
- controllaUtente(String userID) Verifica l'esistenza di un userID già registrato.
- registrazione(...) Inserisce un nuovo utente nel database con i dati forniti.
- Login(String userID, String password) Autentica un utente verificando userID e password nel database, restituendo un oggetto Utente se valido.

- `getCurrentID()` Restituisce l'ultimo ID utente utilizzato nel database.

Le operazioni di validazione e controllo (es. `controllaCF`, `controllaUtente`) sono operazioni con complessità $O(1)$ o $O(\log n)$ dipendendo dalla presenza di indici sui campi nel database. Le operazioni di inserimento (registrazione) e autenticazione (Login) sono generalmente $O(1)$ lato client per quanto riguarda la chiamata, ma lato server dipendono dall'accesso al database.

4.2.4.8 Classe Valutazione:

La classe Valutazione, parte del package `model`, rappresenta le valutazioni che un utente può assegnare a un libro nel sistema BookRecommender. Gestisce diversi criteri di valutazione come stile, contenuto, gradevolezza, originalità ed edizione, oltre al voto finale complessivo. Fornisce metodi per inserire, aggiornare, cancellare valutazioni, e recuperare liste di valutazioni associate a libri.

Metodi principali:

- `inserisciValutazioneLibro(int userID, int bookID, int stile, int contenuto, int gradevolezza, int originalita, int edizione, int votofinale)` Inserisce una nuova valutazione nel database associata a un utente e un libro.
- `aggiornaValutazioneLibro(int userID, int bookID, int stile, int contenuto, int gradevolezza, int originalita, int edizione, int votofinale)` Aggiorna una valutazione esistente per un determinato utente e libro.
- `cancellaValutazioneLibro(int userID, int bookID)` Cancella una valutazione associata a un utente e un libro.
- `getValutazioni(int bookID)` Restituisce tutte le valutazioni associate a un libro.
- `getValutazioneMedia(int bookID, String criterio)` Calcola la media per un criterio specifico (es. stile, contenuto) per tutte le valutazioni di un libro.
- `getValutazioneMediaGenerale(int bookID)` Calcola la media generale delle valutazioni per un libro.

I metodi di inserimento, aggiornamento e cancellazione sono operazioni di manipolazione del database con complessità costante $O(1)$ lato client.

I metodi di recupero dati (`getValutazioni`) hanno complessità lineare $O(m)$ rispetto al numero di valutazioni ritornate.

I calcoli delle medie su criteri specifici (`getValutazioneMedia`) sono anch'essi $O(m)$, dovendo iterare tutte le valutazioni per aggregare i dati.

4.2.5 Package View:

4.2.5.01 Classe `aggiungiLibroController`:

La classe `aggiungiLibroController` gestisce la funzionalità dell'interfaccia utente per aggiungere un libro alla libreria personale dell'utente.

Ecco un riassunto delle sue funzioni principali:

- Costruisce la vista grafica con JavaFX, mostrando:
 - Elenco delle librerie dell'utente
 - Campo di ricerca per il titolo del libro
 - Lista dei risultati di ricerca
 - Pulsanti per cercare il libro, aggiungerlo alla libreria selezionata o tornare indietro
- Permette di cercare un libro per titolo tramite il servizio client in modo asincrono, con task separati per evitare blocchi UI
- Carica le librerie associate all'utente corrente e le mostra nella lista librerie
- Consente di selezionare una libreria e un libro trovato per aggiungerlo alla libreria, dopo aver controllato che il libro non sia già presente

- Mostra messaggi di alert per informazione, errore o mancanza di input per migliorare l'esperienza utente

4.2.5.02 Classe *aggiungiSuggerimentoController*:

La classe `aggiungiSuggerimentoController` gestisce l'interfaccia utente per l'inserimento di suggerimenti di libri all'interno delle librerie personali di un utente. In sintesi, svolge le seguenti funzioni:

- Costruisce la UI con JavaFX mostrando:
 - L'elenco delle librerie dell'utente
 - La lista dei libri nella libreria selezionata
 - Un campo di testo per inserire il titolo del libro da suggerire
 - Pulsanti per inviare il suggerimento o tornare indietro
- Carica dinamicamente le librerie dell'utente e i libri di ciascuna libreria selezionata, gestendo casi di errore e liste vuote
- Permette la selezione di un libro dalla lista per poi inviare un suggerimento collegato a quel libro, controllando preventivamente che non sia stato superato un limite massimo di suggerimenti per quel libro
- Esegue tutte le operazioni in modo asincrono per non bloccare l'interfaccia utente
- Fornisce feedback all'utente tramite alert informativi o di errore in base al successo o fallimento dell'invio del suggerimento

4.2.5.03 Classe *aggiungiValutazioneController*:

La classe `aggiungiValutazioneController` gestisce l'interfaccia utente per l'inserimento di valutazioni dettagliate sui libri da parte degli utenti.

Ecco un riassunto delle sue funzionalità principali:

- Costruisce dinamicamente la UI JavaFX con:
 - Lista delle librerie personali dell'utente
 - Lista dei libri nella libreria selezionata
 - Campo per inserire vari criteri di valutazione numerici (stile, contenuto, gradevolezza, originalità, edizione) da 1 a 5
 - Campi opzionali per note testuali su ogni criterio e una nota complessiva
 - Pulsanti per inviare la valutazione o tornare indietro
- Carica asincronamente le librerie e i libri selezionati dall'utente tramite chiamate al `clientService`
- Effettua controlli di validità sui campi numerici, assicurandosi che siano interi compresi tra 1 e 5 e limita la lunghezza delle note a 256 caratteri
- Calcola il voto finale come media dei valori numerici inseriti
- Invio asincrono della valutazione completa al server tramite `clientService`
- Mostra all'utente messaggi di conferma o errore in base all'esito dell'invio

4.2.5.04 Classe *creaLibreriaController*:

La classe `creaLibreriaController` gestisce l'interfaccia utente e la logica per la creazione di una nuova libreria personale. Principali funzionalità:

- Costruisce la UI con componenti JavaFX per inserire:
 - Il nome della nuova libreria
 - Titoli di libri da aggiungere alla libreria
 - Lista dei libri previsti nella nuova libreria

- Consente di aggiungere titoli di libri alla lista preliminare verificandone l'esistenza tramite chiamate asincrone al servizio client
- Controlla che il nome della libreria sia unico per l'utente prima di creare la libreria
- Esegue la creazione della libreria e l'aggiunta di tutti i libri selezionati tramite task asincroni per non bloccare l'interfaccia utente
- Fornisce all'utente messaggi di avviso o errore per guidare nella procedura (campo mancante, libro già aggiunto, creazione fallita o riuscita)
- Permette di tornare indietro alla vista di login

4.2.5.05 Classe *loginController*:

La classe `loginController` gestisce l'interfaccia utente per il login. Riassunto delle sue funzioni principali:

- Crea dinamicamente la UI con JavaFX, mostrando:
 - Campo di testo per l'inserimento dello username
 - Campo password per l'inserimento della password
 - Pulsante per inviare il login
 - Pulsante per tornare indietro al menu principale
- Gestisce il processo di login tramite il metodo `performSearch()` che:
 - Controlla che i campi non siano vuoti
 - Esegue una chiamata asincrona al servizio client per autenticare l'utente
 - Mostra messaggi di errore se login fallito o campi non compilati
 - Se login riuscito, imposta l'utente loggato in `mainApp` e passa al controller di login effettivo (`trueLoginController`)
- Gestisce messaggi di alert di informazione ed errore per guidare l'utente

4.2.5.06 Classe *registrazioneController*:

La classe `registrazioneController` gestisce l'interfaccia utente e la logica per la registrazione di nuovi utenti. In sintesi, svolge le funzioni principali di:

- Costruzione della UI con JavaFX, mostrando campi per inserire dati di registrazione come nome, cognome, codice fiscale, email, username e password
- Validazione dei dati inseriti, inclusa la verifica che il codice fiscale sia corretto e il controllo che username e codice fiscale non siano già presenti nel database
- Invio della richiesta di registrazione al backend tramite il `clientService`
- Gestione di messaggi di feedback all'utente, sia di errore che di successo, tramite finestre di alert
- Permette all'utente di tornare indietro alla schermata precedente
- Assicura che tutte le operazioni di rete siano gestite in modo asincrono per mantenere reattiva la UI

4.2.5.07 Classe *repositoryController*:

La classe `repositoryController` gestisce l'interfaccia utente e la logica per visualizzare e gestire i libri presenti nel repository. In sintesi, svolge le seguenti funzioni:

- Costruisce la UI con JavaFX mostrando:
 - Una lista di libri ottenuti dal repository
 - Pulsanti per effettuare ricerche e visualizzare dettagli dei libri
- Fornisce metodi per cercare libri nel repository e mostrare i risultati
- Permette la selezione di un libro dall'elenco
- Gestisce messaggi all'utente per casi di errore o informazioni

4.2.5.08 Classe *ricercaAutoreAnnoController*:

La classe *ricercaAutoreAnnoController* gestisce l'interfaccia utente per la ricerca di libri in base all'autore e all'anno di pubblicazione. Principali funzionalità:

- Costruisce la UI con JavaFX mostrando:
 - Campi di testo per inserire autore e anno
 - Pulsante per avviare la ricerca
 - Pulsante per tornare alla schermata precedente
 - Lista per visualizzare i risultati della ricerca
- Controlla che i campi autore e anno non siano vuoti; mostra avvisi se lo sono
- Esegue la ricerca in modo asincrono tramite chiamata al servizio client
- Mostra i risultati trovati nella lista, o un messaggio di nessun risultato
- Gestisce errori di esecuzione mostrando una finestra di alert

4.2.5.09 Classe *ricercaAutoreController*:

La classe *ricercaAutoreController* gestisce l'interfaccia utente e la logica per la ricerca di libri per autore. Riassunto delle funzionalità:

- Costruisce la UI con JavaFX mostrando:
 - Campo per inserire il nome dell'autore da cercare
 - Pulsante per avviare la ricerca
 - Pulsante per tornare alla schermata principale
 - Lista per visualizzare i risultati della ricerca
- Controlla che il campo autore non sia vuoto, altrimenti mostra un messaggio di avviso
- Esegue in modo asincrono la ricerca
- Mostra i risultati trovati nella lista o un messaggio se non sono stati trovati libri
- Gestisce errori di ricerca mostrando messaggi di alert all'utente

4.2.5.10 Classe *ricercaInfoController*:

La classe *ricercaInfoController* gestisce l'interfaccia utente per effettuare ricerche di libri basate su informazioni generiche. Riassunto delle funzionalità:

- Costruisce una interfaccia grafica semplice con JavaFX, comprendente:
 - Un campo di testo dove l'utente inserisce la parola chiave o frase da cercare
 - Un pulsante per avviare la ricerca
 - Una lista per mostrare i risultati trovati
 - Un pulsante per tornare alla schermata precedente
- Al click del pulsante di ricerca, verifica che il campo non sia vuoto
- Esegue la ricerca in maniera asincrona chiamando il servizio client con la parola chiave
- Mostra i libri trovati nella lista risultati o avvisa se la ricerca non ha prodotto risultati
- Gestisce la comunicazione e gli errori mostrando finestre di alert informative o di errore per l'utente

4.2.5.11 Classe *ricercaTitoloController*:

La classe *ricercaTitoloController* gestisce l'interfaccia utente e la logica per la ricerca di libri per titolo. Principali funzionalità:

- Costruisce la UI con JavaFX mostrando:
 - Campo di testo per inserire un titolo da ricercare
 - Pulsante per avviare la ricerca

- Pulsante per tornare alla schermata principale
- Lista per visualizzare i risultati
- Controlla che il campo titolo non sia vuoto, altrimenti mostra un messaggio di avviso
- Esegue la ricerca in modo asincrono tramite chiamata al servizio client
- Visualizza i libri trovati nella lista risultati o un messaggio di assenza risultati
- Gestisce eventuali errori mostrando notifiche all'utente

4.2.5.12 Classe *trueLoginController*:

La classe *trueLoginController* gestisce l'interfaccia principale e il menu di navigazione dell'app una volta che l'utente ha effettuato il login con successo. Riassunto delle sue funzionalità principali:

- Costruisce la UI con JavaFX mostrando un pannello con pulsanti per accedere alle funzionalità principali:
 - Creare una nuova libreria personale
 - Aggiungere un libro a una libreria
 - Inserire una valutazione per un libro
 - Inserire un suggerimento di lettura
 - Uscire dall'applicazione
- Ogni pulsante è associato a un metodo che mostra il controller e la vista corrispondente per la funzionalità scelta
- Gestisce anche il comando per uscire dall'applicazione
- Coordinamento principale fra i diversi controller e vista dopo il login

4.2.5.13 Classe *viewInfoController*:

La classe *viewInfoController* gestisce l'interfaccia utente per visualizzare informazioni dettagliate di un libro. Riassunto delle funzionalità principali:

- Costruisce la UI con JavaFX mostrando:
 - Campo di testo per inserire il titolo di un libro
 - Pulsante per avviare la ricerca
 - Pulsante per tornare alla schermata precedente
 - Lista per mostrare i dettagli del libro trovato
- Controlla che il campo titolo non sia vuoto, altrimenti mostra un messaggio di avviso
- Esegue la ricerca in modo asincrono tramite chiamata al servizio client
- Visualizza le informazioni dettagliate del libro trovato o un messaggio se non sono presenti risultati
- Gestisce errori mostrando messaggi di alert all'utente

4.2.5.14 Classe *viewRatingsController*:

La classe *viewRatingsController* gestisce l'interfaccia utente per visualizzare le valutazioni di un libro. Riassunto delle sue funzionalità:

- Costruisce una UI JavaFX con:
 - Campo di testo per inserire il titolo del libro da cercare
 - Pulsanti per avviare la ricerca e per tornare indietro
 - Lista per visualizzare le valutazioni trovate
- Verifica che il campo titolo non sia vuoto, altrimenti avvisa con un messaggio
- Esegue la ricerca delle valutazioni in modo asincrono
- Mostra le valutazioni trovate o un messaggio se non sono presenti risultati
- Gestisce gli errori e mostra allarmi in caso di problemi durante la ricerca

4.2.5.15 Classe *viewSuggestionController*:

La classe `viewSuggestionController` gestisce l'interfaccia utente per visualizzare suggerimenti di lettura relativi a un libro. Riassunto delle funzionalità:

- Costruisce una UI con JavaFX che include:
 - Un campo di testo per inserire il titolo del libro
 - Un pulsante per avviare la ricerca dei suggerimenti
 - Un pulsante per tornare alla schermata principale del repository
 - Una lista per visualizzare i risultati dei suggerimenti trovati
- Controlla che il campo titolo non sia vuoto, mostrando un messaggio di avviso altrimenti
- Esegue la ricerca dei suggerimenti in modo asincrono
- Visualizza i suggerimenti trovati nella lista o mostra un messaggio se non sono presenti suggerimenti
- Gestisce errori durante la ricerca comunicandoli all'utente tramite finestre di alert

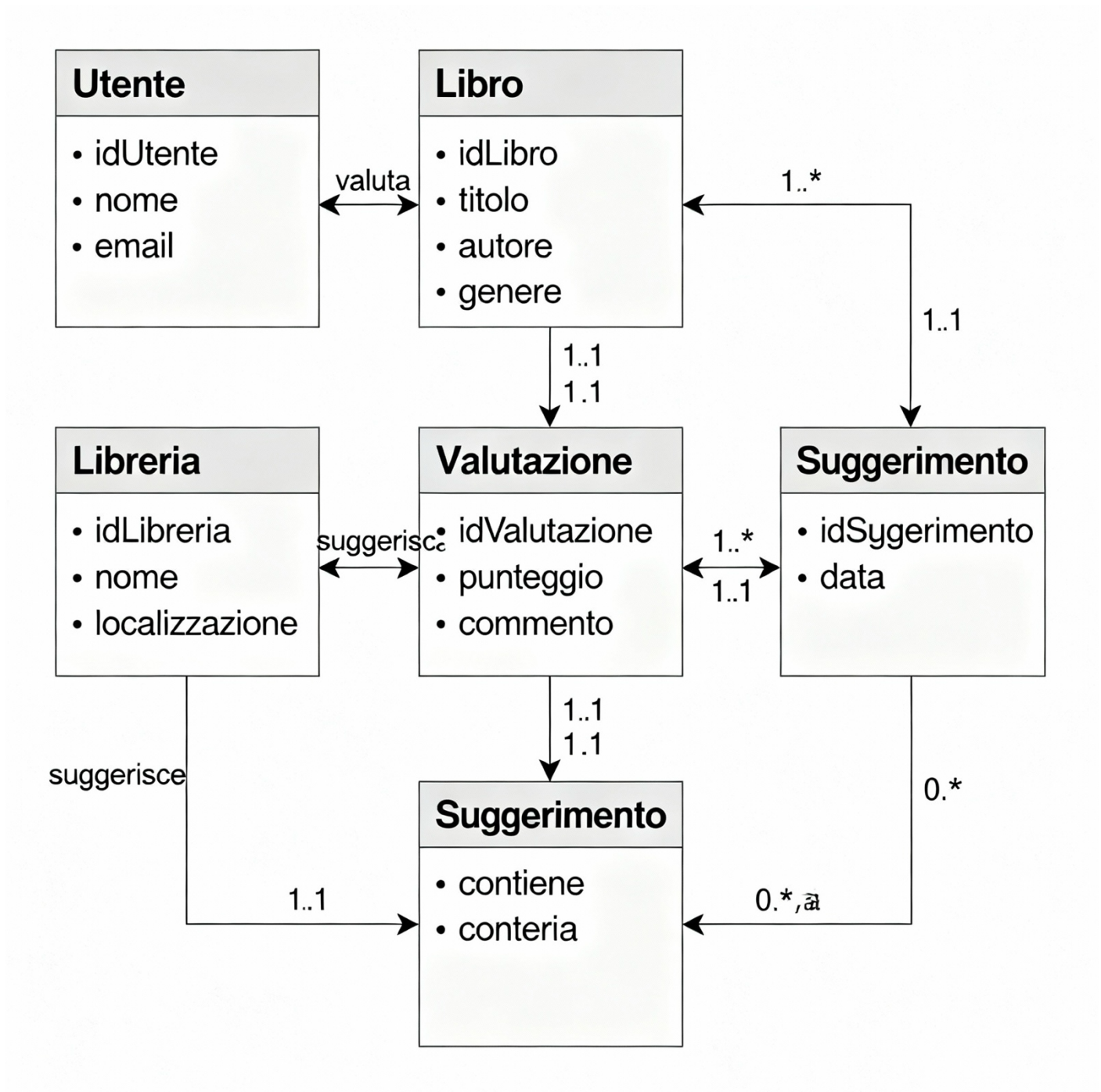
4.2.5.16 Classe *visualizzaInfoController*:

La classe `visualizzaInfoController` gestisce l'interfaccia utente per visualizzare informazioni dettagliate di un libro. Riassunto:

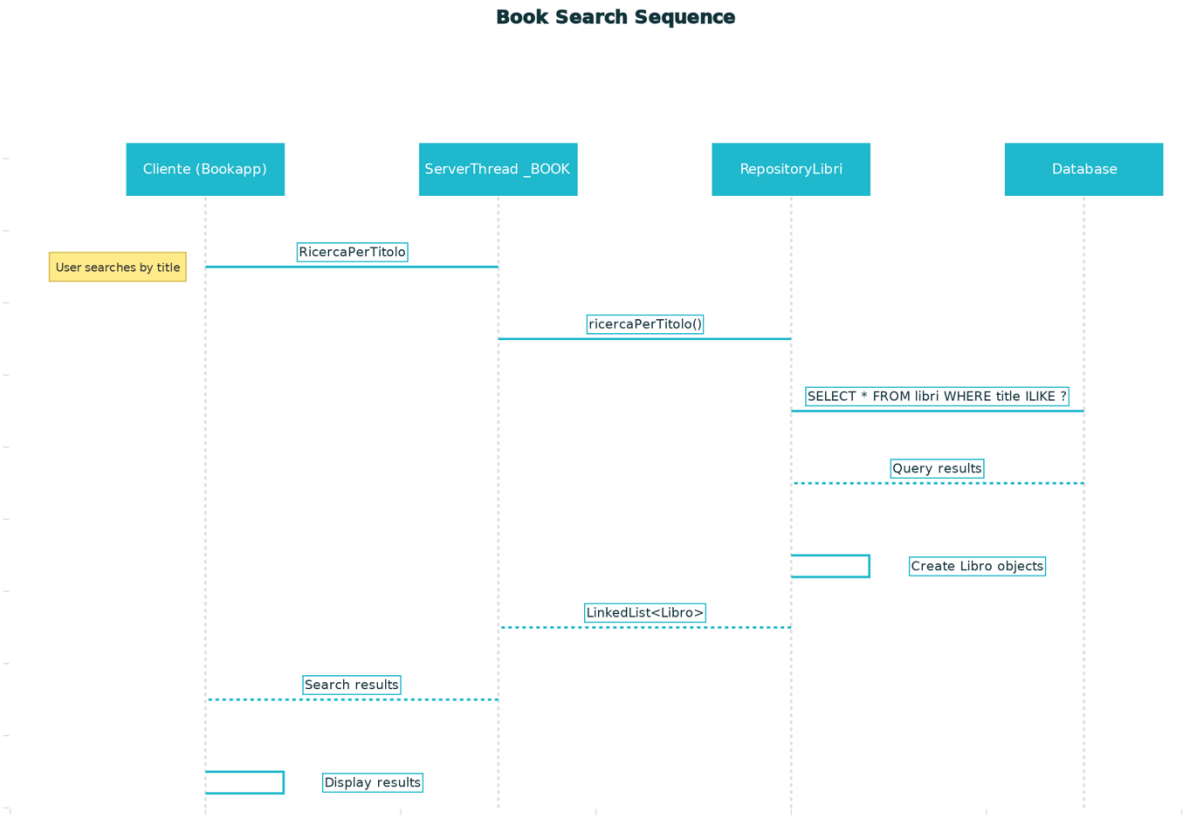
- Costruisce la UI con JavaFX mostrando:
 - Un campo di testo per inserire il titolo del libro
 - Un pulsante per avviare la ricerca
 - Un pulsante per tornare alla schermata precedente
 - Una lista per mostrare i dettagli del libro trovato
- Al click del pulsante, controlla che il campo titolo non sia vuoto, altrimenti avvisa l'utente
- Esegue una ricerca asincrona chiamando il client service per ottenere le informazioni del libro
- Visualizza nella lista i dettagli del libro o un messaggio se non vengono trovati dati
- Gestisce errori con messaggi di alert per l'utente

4.3 Diagrammi UML

4.3.1 Class diagram

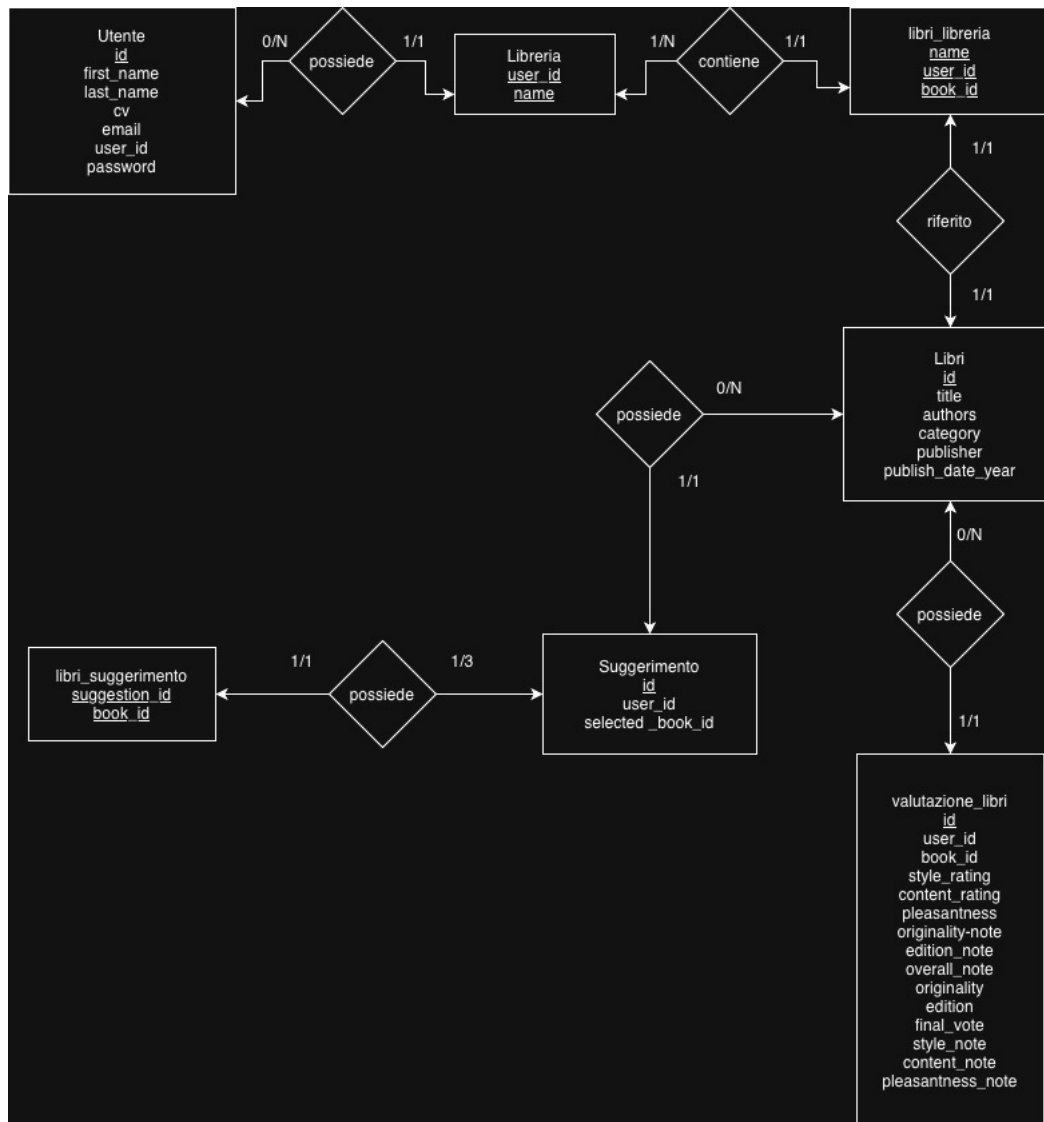


4.3.2 Sequence diagram della ricerca per titolo



5. Progettazione database

5.1 Diagramma ER



Cardinalità utente – libreria = 1/N perché un utente registrato può avere più librerie mentre una libreria è posseduta da un solo utente.

Cardinalità libreria – libri_libreria = 1/N perché una libreria contiene più libri e un libro è associato ad una sola libreria.

Cardinalità libri_libreria – libri = 1/1 perché ogni libro è associato ad un'unica libreria e viceversa.

Cardinalità libri – valutazione_libri = 1/N perché ogni libro può avere più valutazioni mentre ogni valutazione è riferita ad un solo libro.

Cardinalità libri – suggerimento = 1/N perché ogni libro può avere più suggerimenti mentre un suggerimento è associato ad un solo libro.

Cardinalità suggerimento – libri_suggerimento = 1/3 perché possono essere suggeriti massimo 3 libri per suggerimento.

5.2 Architettura database

Il database del sistema Book Recommender in PostgreSQL contiene sette entità principali:

- **libri**: contiene tutti i dati relativi ai libri (id, titolo, autori, categoria, editore, anno di pubblicazione).
- **utente_registrati**: memorizza utenti registrati con attributi anagrafici e di accesso (id, nome, cognome, codice fiscale, email, user_id, password).
- **libreria**: rappresenta una raccolta personale di libri di uno specifico utente (user_id, nome della libreria).
- **libri_libreria**: tabella di relazione molti-a-molti tra librerie e libri (user_id, nome libreria, book_id).
- **valutazioni_libri**: registra tutte le valutazioni degli utenti su ciascun libro, includendo voti numerici e note testuali su più criteri (id, user_id, book_id, style_rating, content_rating, pleasantness, originality, edition, final_vote).
- **suggerimento**: gestisce le raccomandazioni, collegando ogni suggerimento a un utente e a un libro di partenza (id, user_id, selected_book_id).
- **libri_suggerimento**: permette l'associazione di più libri suggeriti a ciascun suggerimento

Le relazioni tra le entità sono tutte vincolate da chiavi esterne, con le opportune regole di integrità referenziale (cancellazione in cascata dove necessario). Un utente può avere più librerie, ogni libreria può contenere più libri, ogni libro può essere valutato più volte e suggerito in più raccomandazioni. Il modello garantisce normalizzazione e facilità di estensione.

5.3 Schema relazionale

Utente

(id PRIMARY KEY, first_name, last_name, cv, email, user_id, password)

Libreria

(user_id REFERENCES Utente(id), name, PRIMARY KEY(user_id, name))

Libri

(id PRIMARY KEY, title, authors, category, publisher, publish_date_year)

Libri_Libreria

(name, user_id, book_id REFERENCES Libri(id), PRIMARY KEY(name, user_id, book_id), FOREIGN KEY(user_id, name) REFERENCES Libreria(user_id, name))

Suggerimento

(id PRIMARY KEY, user_id REFERENCES Utente(id), selected_book_id REFERENCES Libri(id))

Libri_Suggerimento

(suggestion_id REFERENCES Suggerimento(id), book_id REFERENCES Libri(id), PRIMARY KEY(suggestion_id, book_id))

Valutazione_Libri

(id PRIMARY KEY, user_id REFERENCES Utente(id), book_id REFERENCES Libri(id), style_rating, content_rating, pleasantness, originality_note, edition_note, overall_note, originality, edition, final_vote, style_note, content_note, pleasantness_note)

6. Limiti della soluzione sviluppata

6.1 Limiti funzionali e di sicurezza

6.1.1 Memorizzazione Password in chiaro

Le password degli utenti sono memorizzate nel database in chiaro senza hashing o encryption. Questo rappresenta una vulnerabilità critica.

```
// METODO INSICURO ATTUALE:  
statement.setString(7, password); // Password in chiaro  
  
// RACCOMANDAZIONE:  
String hashedPassword = BCrypt.hashpw(password, BCrypt.gensalt());
```

6.1.2 Credenziali hardcoded

Le credenziali del database sono inserite direttamente nel codice:

```
String urlDefaultDB = "jdbc:postgresql://" + host + ":" + PORT + "/postgres";  
DriverManager.getConnection(urlDefaultDB, userSQL, "LaboratorioB");
```

Dovrebbero essere esternalizzate in file di configurazione.

6.1.3 Mancanza di caching

Non ci sono meccanismi di cache. Se un utente ricerca lo stesso libro più volte, il database è interrogato ogni volta.

6.2 Limiti di usabilità

6.2.1 Messaggi di errore generici

Gli errori sono mostrati genericamente senza contesto utile.

6.2.2 Assenza di validazione client-side

Tutto è validato lato server, causando latenza nel feedback.

6.2.3 Interfaccia non-responsive

Durante le query lunghe, l'interfaccia JavaFX potrebbe bloccarsi (operazioni JDBC sincrone sul EDT).

7. Sitografia e bibliografia

- Librerie e Framework:

Apache Maven - <https://maven.apache.org/>

JavaFX 11 - <https://openjfx.io/>

PostgreSQL JDBC Driver - <https://jdbc.postgresql.org/>

JUnit 5 - <https://junit.org/junit5/>

-Standard Database:

- SQL Standard - ISO/IEC 9075

PostgreSQL Documentation - <https://www.postgresql.org/docs/>